

Niveau : III Date : 07/11/2024 Durée : 2 H Nombre de pages : 2	Documents : Non autorisés Calculatrice : Non autorisée Annexe : Non Barème à titre indicatif : (9.5+10,5)	 École Nationale des Sciences de l'Informatique Université de la Manouba A.U. 2024/2025
Enseignant(s) : D. DHOUIB, I. AMDOUNI, N. DRIDI, M. BEN SASSI, R. CHEBIL		Réf : ID.104 Version :01
DEVOIR SURVEILLÉ Algorithmique et Structures de Données		

Exercice 1 (9.5 pts)

Soit une matrice $N \times M$ remplie de 0 et de 1. La taille maximale de la matrice ne peut pas dépasser 50×50 . Le but est de déterminer s'il existe un chemin de la cellule en haut à gauche de coordonnées (1,1) à la cellule en bas à droite de coordonnées (N, M). On peut se déplacer uniquement vers le bas ↓, vers le haut ↑, vers la droite → ou vers la gauche ←. On ne peut passer que par les cellules contenant 1, et on ne peut visiter la même cellule qu'une seule fois. Ainsi, une cellule est caractérisée par ses coordonnées ainsi que son état. **La cellule (1,1) contient toujours la valeur 1.**

Exemple : Soit la matrice (4*4) suivante :

```
1 0 1 1
1 1 0 1
0 1 1 0
1 0 1 1
```

Le programme doit trouver un chemin depuis (1,1) vers (4,4), en passant uniquement par les cellules contenant des 1. Pour cet exemple, un chemin possible serait : (1,1) → (2,1) → (2,2) → (3,2) → (3,3) → (4,3) → (4,4)

1. Proposer la structure de donnée pour représenter une cellule dans une matrice. **(1 pt)**
2. Proposer la structure de donnée pour représenter toute la matrice. **(0.5 pt)**
3. Étant donné la matrice, proposer un **module itératif** Saisie_Matrice qui permet la saisie d'une matrice. Les valeurs saisies doivent être soit 0 soit 1. **(2 pts)**
4. Proposer un **module récursif** Chercher_Chemin qui recherche un chemin dans une matrice donnée et retourne Vrai s'il en trouve, Faux sinon. Indication : Ce module doit vérifier si la cellule courante est dans la matrice, contient 1, et n'a pas encore été visitée. **(2 pts)**

Le système génère une chaîne de caractères décrivant un chemin qui commence à la cellule de départ (1,1) et se termine à la cellule (N, M) dans une matrice. Cette chaîne est composée de lettres représentant les directions : H pour Haut, B pour Bas, D pour Droite, et G pour Gauche, en majuscules ou en minuscules. Par exemple, une chaîne **valide** générée pour décrire un chemin dans la matrice pourrait être : "bDBdHD"

5. Écrivez un module qui, étant donné une chaîne de caractères, vérifie sa validité. Une chaîne est considérée comme valide si elle n'est pas vide et ne contient que les lettres citées ci-dessus. **(1 pt)**
6. Pour enregistrer les cellules de la matrice correspondant au chemin décrit par la chaîne, on se propose d'utiliser une structure de données intermédiaire qui permet un accès rapide et contigu. Quelle structure de données proposez-vous pour cette tâche ? **(1 pt)**

7. Écrivez un module itératif qui parcourt la matrice en suivant les directions décrites par la chaîne de caractères. Ce module doit enregistrer les cellules visitées dans la structure intermédiaire et retourner cette structure. (2 pts)

Exercice 2 (10.5 pts)

NB. L'utilisation d'un tableau intermédiaire dans toutes les questions de cet exercice est interdite

Vous êtes chargé de gérer une collection de poupées russes imbriquées. Chaque poupée a nom, une taille, et elle peut être ouverte ou fermée. Une poupée p1, dite poupée mère, ne peut en contenir une autre poupée p2, dite poupée fille, que si p1 est ouverte et que p2 a une taille plus petite. Ainsi, chaque poupée est caractérisée, en plus des champs cités ci-dessus, par le nom de sa poupée mère et le nom de sa poupée fille respectivement, s'il y en a. Dans le cas où une poupée n'a pas de mère ou de fille, le champ correspondant est la chaîne vide. Initialement, chaque poupée est fermée, n'a pas de mère et n'a pas de fille.

I. Partie I

1. Définir le type `Poupee`. (0.75 pt)
2. Proposer un module `creer_poupee(Poupee)` qui prend en paramètre une poupée et la met dans son état initial et la retourne. (1.25 pts)
On suppose qu'on dispose de deux fonctions `generer_nom` permettant de générer un nom valide de façon aléatoire, et la fonction `generer_taille` qui permet de générer une taille unique en fonction d'un nom donné.
3. Proposer un module `placer_dans` qui place une poupée donnée dans une autre poupée donnée qu'il ouvre. Ce module affiche « Succès » si le placement s'effectue avec succès, et affiche « Echec » sinon. (1.25 pts)
4. Proposer un module `sortir_de` qui permet de fermer et vider une poupée de sa poupée fille. (1 pt)

II. Partie II

Vous utiliserez maintenant un tableau de type `TAB` de taille maximale `MAX` pour stoker `N` poupées. Le but est d'utiliser ce tableau pour imbriquer les poupées, de la façon la plus simple possible, c'est-à-dire les mettre les unes dans les autres en respectant la taille croissante.

On utilisera l'opérateur `=` pour comparer deux chaînes de caractères. Vous pourrez utiliser les modules de la Partie I.

5. Définir le type `TAB`. (0.5 pt)
6. Proposer un module `remplir_tab` qui permet de remplir un tableau donné par `N` poupées créées. (0.75 pt)
7. Proposer un module `chercher_poupee` qui cherche une poupée de nom donné dans le tableau de poupées et retourne l'indice de cette poupée si elle existe sinon retourne -1. (1pt)
8. Dédurre un module `supprimer_poupee` qui supprime une poupée de nom donné du tableau. (1pt)
9. Proposer un module `tri_tab` qui tri un tableau de poupées donné dans l'ordre croissant de leur taille en utilisant l'algorithme de tri par sélection. (1.5 pts)
10. Dédurre le module `imbriquer_poupees` qui permet d'imbriquer de façon simple les poupées remplies dans un tableau donné en les plaçant les unes dans les autres en respectant la taille. (1.5 pts)

Bon Travail